

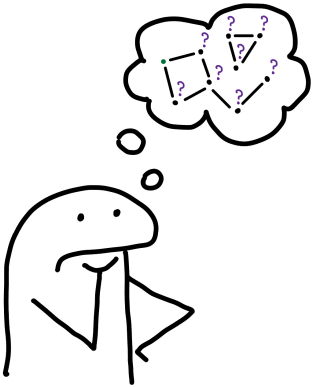
Typed Embedding of miniKanren for Functional Conversion

Igor Engel, **Ekaterina Verbitskaia**

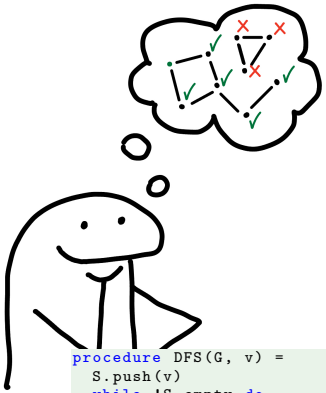
JetBrains Research,
Constructor University, Bremen

October 17, 2025

No miniKanren — One Task, One Program

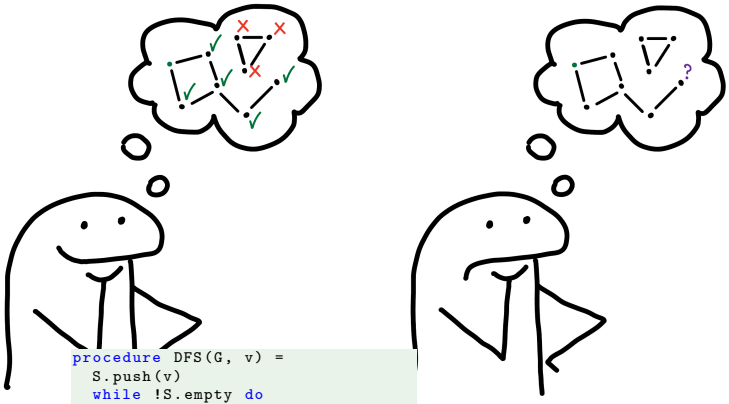


No miniKanren — One Task, One Program



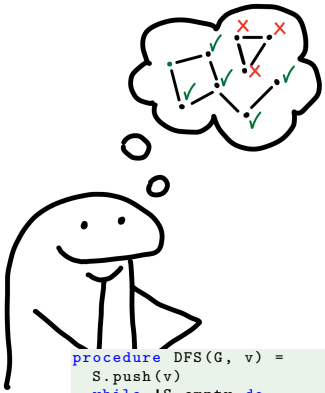
```
procedure DFS(G, v) =
  S.push(v)
  while !S.empty do
    v = S.pop()
    if (!seen[v]) then
      seen.add(v)
      for (_, w) in G.edges(v) do
        S.push(w)
```

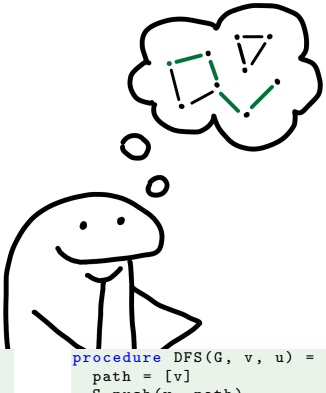
No miniKanren — One Task, One Program



```
procedure DFS(G, v) =
  S.push(v)
  while !S.empty do
    v = S.pop()
    if (!seen[v]) then
      seen.add(v)
      for (_, w) in G.edges(v) do
        S.push(w)
```

No miniKanren — One Task, One Program

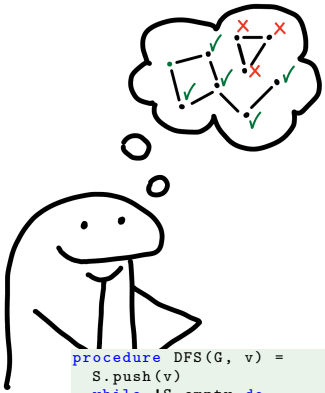




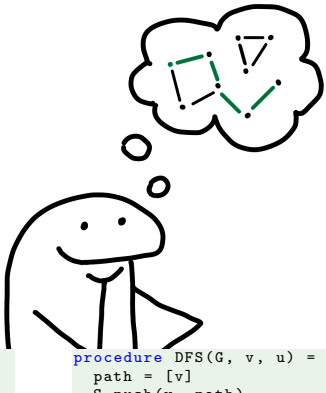
```
procedure DFS(G, v) =
  S.push(v)
  while !S.empty do
    v = S.pop()
    if (!seen[v]) then
      seen.add(v)
      for (_, w) in G.edges(v) do
        S.push(w)
```

```
procedure DFS(G, v, u) =
  path = [v]
  S.push(v, path)
  while !S.empty do
    (v, path) = S.pop()
    if (v == u) then return path
    if (!seen[v]) then
      seen.add(v)
      for (_, w) in G.edges(v) do
        S.push((w, path.add(w)))
  return none
```

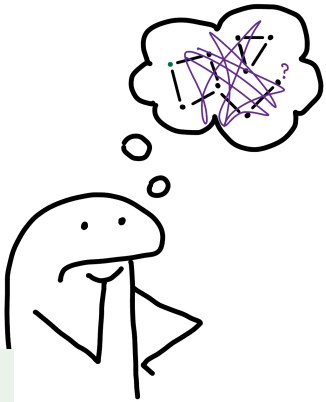
No miniKanren — One Task, One Program



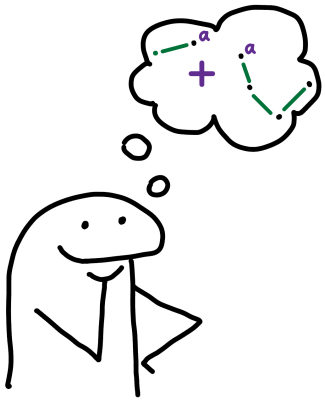
```
procedure DFS(G, v) =
  S.push(v)
  while !S.empty do
    v = S.pop()
    if (!seen[v]) then
      seen.add(v)
      for (_, w) in G.edges(v) do
        S.push(w)
```



```
procedure DFS(G, v, u) =
  path = [v]
  S.push(v, path)
  while !S.empty do
    (v, path) = S.pop()
    if (v == u) then return path
    if (!seen[v]) then
      seen.add(v)
      for (_, w) in G.edges(v) do
        S.push((w, path.add(w)))
    return none
```

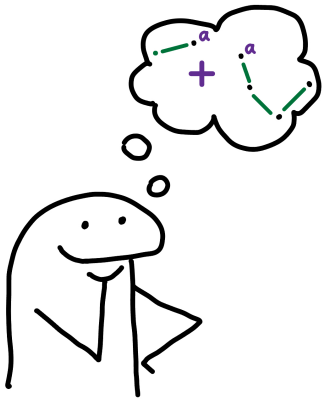


With miniKanren — Many Tasks, One Relation



$$path(v, u) = \begin{cases} [v], & \text{if } v = u \\ v : p, & \text{if } \exists w : edge(v, w) \wedge p = path(w, u) \end{cases}$$

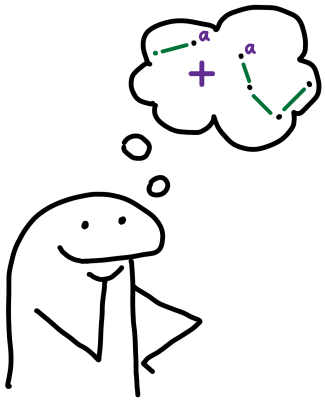
With miniKanren — Many Tasks, One Relation



$$path(v, u) = \begin{cases} [v], & \text{if } v = u \\ v : p, & \text{if } \exists w : edge(v, w) \wedge p = path(w, u) \end{cases}$$

```
let rec path^o v u p =                                  (* relation *)
  (v ≡ u ∧ p ≡ [v]) ∨
  (fresh (w p1)
    (edge^o v w ∧
     path^o w1 u p1 ∧
     p ≡ v : p1) )
```

With miniKanren — Many Tasks, One Relation

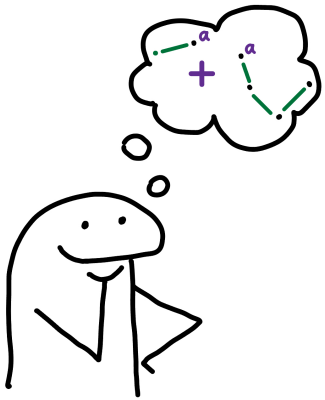


$$path(v, u) = \begin{cases} [v], & \text{if } v = u \\ v : p, & \text{if } \exists w : edge(v, w) \wedge p = path(w, u) \end{cases}$$

```
let rec path^o v u p =                                (* relation *)
  (v ≡ u ∧ p ≡ [v]) ∨
  (fresh (w p₁)
    (edge^o v w ∧
     path^o w₁ u p₁ ∧
     p ≡ v : p₁) )
```

```
run 1 q (path^o A E q)
```

With miniKanren — Many Tasks, One Relation



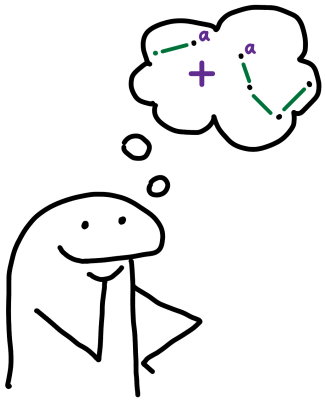
$$path(v, u) = \begin{cases} [v], & \text{if } v = u \\ v : p, & \text{if } \exists w : edge(v, w) \wedge p = path(w, u) \end{cases}$$

```
let rec path^o v u p =                                  (* relation *)
  (v ≡ u ∧ p ≡ [v]) ∨
  (fresh (w p₁)
    (edge^o v w ∧
     path^o w₁ u p₁ ∧
     p ≡ v : p₁) )
```

```
run 1 q (path^o A E q)
```

```
run* q (path^o A E q)
```

With miniKanren — Many Tasks, One Relation



$$path(v, u) = \begin{cases} [v], & \text{if } v = u \\ v : p, & \text{if } \exists w : edge(v, w) \wedge p = path(w, u) \end{cases}$$

```
let rec path^o v u p =                                  (* relation *)
  (v ≡ u ∧ p ≡ [v]) ∨
  (fresh (w p₁)
    (edge^o v w ∧
     path^o w₁ u p₁ ∧
     p ≡ v : p₁) )
```

```
run 1 q (path^o A E q)
```

```
run* q (path^o A E q)
```

```
run* q (fresh (p) (path^o A q p))
```

Solvers from Verifiers

Verifiers




```
let rec is_path v u path =                    (* function *)
  match path with
  | [v'] → v == v' && v == u
  | v' :: w :: t →
      if v == v' && edge v' w
      then is_path w u (w :: t)
      else false
```

Solvers




```
let rec dfs ... =
  for ...
    if seen ...
      ... dfs ...
    ... return ...
```

Solvers from Verifiers




```
let rec is_path v u path =                    (* function *)
  match path with
  | [v'] → v == v' && v == u
  | v' :: w :: t →
      if v == v' && edge v' w
      then is_path w u (w :: t)
      else false
```

relational conversion

```
let rec path° v u p =                         (* relation *)
  (v ≡ u ∧ p ≡ [v]) ∨
  (fresh (w p₁)
    (edge° v w ∧
     path° w₁ u p₁ ∧
     p ≡ v : p₁) )
```

verifier

solver

Solvers from Verifiers: Examples




```
let rec eval° st fm u =
  fresh (x y v w z)
    (fm ≡ Conj x y ∧
     eval° st x v ∧
     eval° st y w ∧
     and° v w u) ∨
     ...
```

→ evaluation

→ program generation

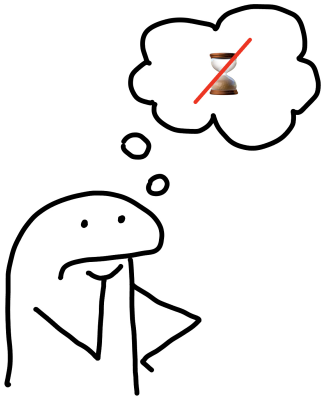
```
let rec type° e t =
  (fresh (x y)
    (e ≡ Int x ∧ t = TInt) ∨
    (e ≡ Plus x y ∧
     type° x Int ∧
     type° y Int ∧
     t ≡ Int)) ∨
     ...
```

→ type checking

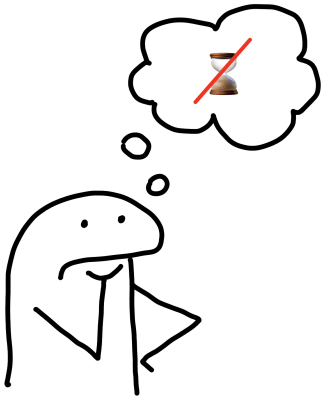
→ type inference

→ type inhabitance

The Issue



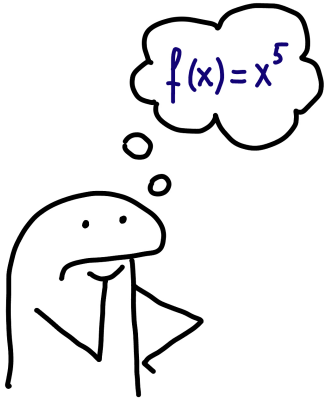
The Issue



- Unifications are expensive
- The order of conjunctions is finicky
- But! We know something that can help

Specialization

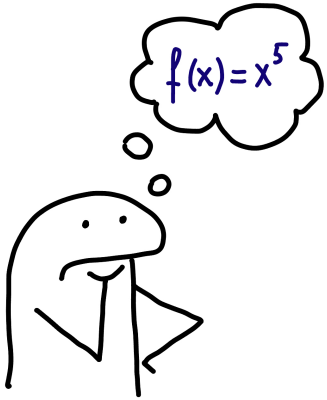
Specialization




```
let rec power base exp =
  if exp == 0
  then 1
  else base * power base (exp - 1)
```

```
let rec power_5 base =                          (* power base 5 *)
  base * base * base * base * base
```

Specialization

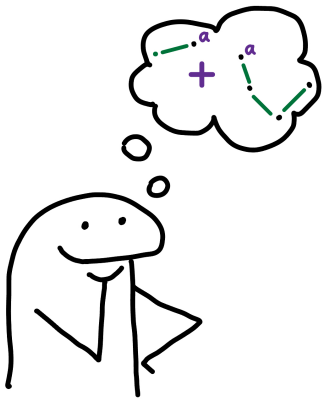


$$program : I_{static} \times I_{dynamic} \to O$$

$$program_{I_{static}} : I_{dynamic} \to O$$

Same outputs for the same inputs

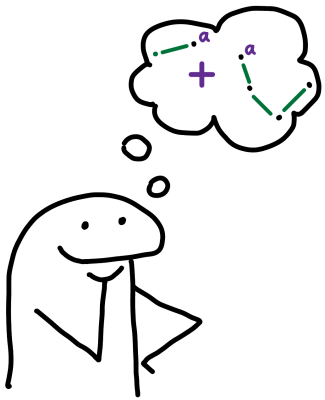
Static Data in miniKanren: Modes



First two arguments known (mode `in in out`)

```
run* q (path^o A E p)
```

Static Data in miniKanren: Modes



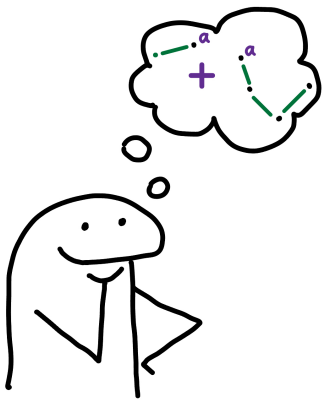
First two arguments known (mode `in in out`)

```
run* q (path^o A E p)
```

Last argument known (mode `out out in`)

```
run* (p q) (path^o p q [A, B, C, D, E])
```

Static Data in miniKanren: Modes



First two arguments known (mode `in in out`)

```
run* q (path^o A E p)
```

Last argument known (mode `out out in`)

```
run* (p q) (path^o p q [A, B, C, D, E])
```

Arguments may not be known (mode `in out out`)

```
run* (q) (fresh (p) (path^o A q p))
```

Optimal Conjunction Order Depends on a Mode

```
let rec sort^o x y =
  (x ≡ [] ∧ y ≡ []) ∨
  (fresh (s xs xs1)
    y ≡ s :: xs1 ∧
    smallest^o x s xs ∧
    sort^o xs xs1)
```

✓ sorting

🕒 permutations

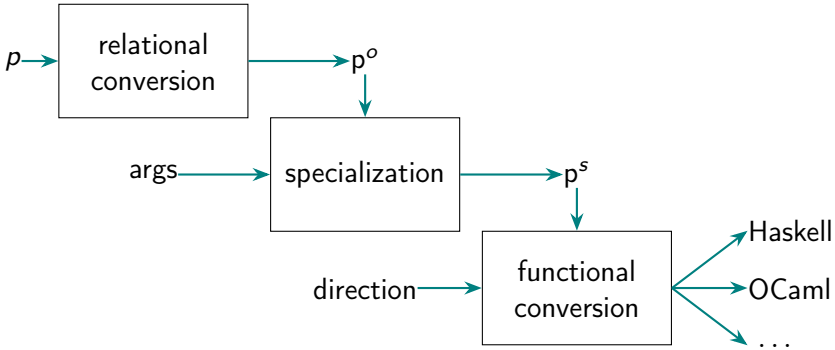
```
let rec sort^o x y =
  (x ≡ [] ∧ y ≡ []) ∨
  (fresh (s xs xs1)
    y ≡ s :: xs1 ∧
    sort^o xs xs1 ∧
    smallest^o x s xs)
```

🕒 sorting

✓ permutations

Functional Conversion

Generating Solvers from Verifiers



Addition in the Backward Direction: Nondeterminism

```
let rec add^o x y z =
  (x ≡ O ∧ y ≡ z) ∨
  (fresh (x₁ z₁)
    (x ≡ S x₁ ∧
     add^o x₁ y z₁ ∧
     z ≡ S z₁) )
```

```
addOOI :: Nat → Stream (Nat, Nat)
addOOI z =
  return (O, z) <|>
  case z of
    O → Empty
    S z₁ → do
      (x₁, y) ← addOOI z₁
      return (S x₁, y)
```

$\text{add}^{\circ}\ x\ y\ 1 = [\{x \mapsto 0, y \mapsto 1\}, \{x \mapsto 1, y \mapsto 0\}]$

addOOI 1 = [(0,1), (1,0)]

Free Variables in Answers: Generators

```
let rec add^o x y z =
  (x ≡ O ∧ y ≡ z) ∨
  (fresh (x₁ z₁)
    (x ≡ S x₁ ∧
     add^o x₁ y z₁ ∧
     z ≡ S z₁) )
```

$\text{add}^\circ\ 1\ y\ z = \{z \mapsto S\ y\}$

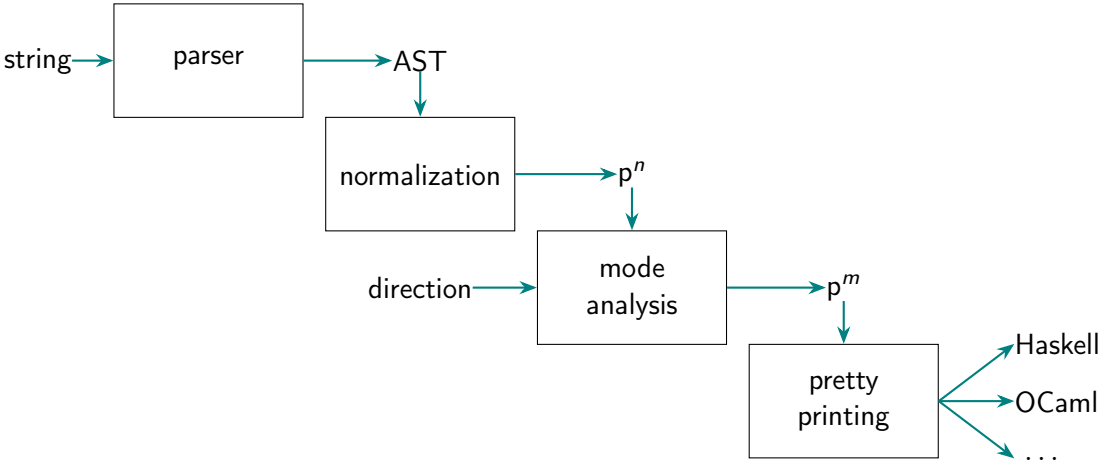
$\text{genNat} = [0,\ 1,\ 2,\ \ldots]$

$\text{addIOO}\ 1 = [(0,1),\ (1,2),\ (2,3),\ \ldots]$

```
addIOO :: Nat → Stream (Nat, Nat)
addIOO x =
  case x of
    O → do
      z ← genNat
      return (z, z)
    S x₁ → do
      (y, z₁) ← addIOO x₁
      return (y, S z₁)

genNat :: Stream Nat
genNat =
   (return O) <|> (S <$> genNat)
```

Old Functional Conversion



Untyped Deep Embedding

- No data type definitions
  - Constructors start with an Uppercase letter
- No static checks
  - It doesn't even check that the symbol is defined
- No extensibility

```
addo x y z =
  (x == O & y == z) |
  (fresh x', z' in
    x == S x' & z == S z' & addo x' y z'
  );

? addo (S (S O)) y z
```

The Awkward Parts of the Old Functional Conversion

A monolithic Term type

```
data Term = O
          | S Term
          | Nil
          | Cons Term Term
          | ...
```

The Awkward Parts of the Old Functional Conversion

A monolithic Term type

```
data Term = O
          | S Term
          | Nil
          | Cons Term Term
          | ...
```

Explicit passing of generators

```
addIOO :: Nat → Stream (Nat, Nat)
addIOO x genNat =
  case x of
    O → do
      z ← genNat
      return (z, z)
    ....
```

Typed Shallow Embedding

The Requirements

- Typed terms
  - Get rid of the monolithic Term type
  - Implicit generator passing
- Shallow embedding
  - Extensibility of miniKanren itself
  - Take advantage of the Haskell type system
- Access to the AST
  - Reordering of conjuncts
  - Unification inspection
  - Access to relation names and variables

Example

```
addoRel :: (MiniKanren rel var)
        => Logic Nat var → Logic Nat var → Logic Nat var → Relation rel
addoRel = relation3 "addo" (\x y z → asum
    [ do
        x <=> zero
        y <=> z
    , fresh2 (\x' z' → do
        z <=> succ z'
        x <=> succ x'
        call (addoRel x' y z'))
    ])
```

Logic Term Type

```
class LogicType a where -- a - underlying type
  data WithLogic a (var :: Type → Type) :: Type -- logic type
  ...

data Logic a var = Free (Var a var) | Ground (WithLogic a var)
data Nat = Z | S Nat
data WithLogic Nat var = LogicZ | LogicS (Logic Nat var)
```

Final-Tagless Style

```
class ExprSYM repr where
  lit :: Int → repr
  neg :: repr → repr
  add :: repr → repr → repr

instance ExprSYM Int where
  lit n = n
  neg e = -e
  add x y = x + y

instance ExprSYM AST where
  lit n = Lit n
  neg e = Neg e
  add x y = Add x y
```

Final-Tagless Style

```
class ExprSYM repr where
  lit :: Int → repr
  neg :: repr → repr
  add :: repr → repr → repr

instance ExprSYM Int where
  lit n = n
  neg e = -e
  add x y = x + y

instance ExprSYM AST where
  lit n = Lit n
  neg e = Neg e
  add x y = Add x y
```

```
class MulSYM repr where
  mul :: repr → repr → repr

instance MulSYM Int where
  mul x y = x * y

instance MulSYM AST where
  mul x y = Mul x y
```

Kanren Type Class

```
class (Alternative rel, Functor (KVar rel)) => Kanren rel where
  -- Variable representation for the interpreter
  data KVar rel :: Type → Type

  -- Core language
  (≡) :: (LogicType a) => Logic a (KVar rel) → Logic a (KVar rel) → rel ()
  call :: Relation rel → rel ()
  fresh :: (LogicType t) => (Var t (KVar rel) → rel a) → rel a
```

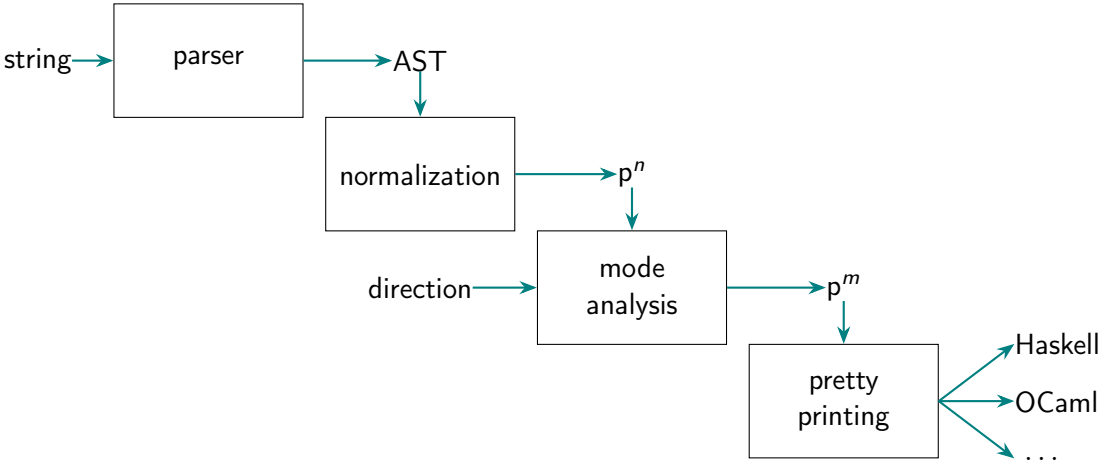
- Disjunction is expressed by asum from Alternative
- Conjunction is not monadic, it's applicative
  - Allows reordering the conjuncts
  - {−# Language ApplicativeDo #−}

Interpreters

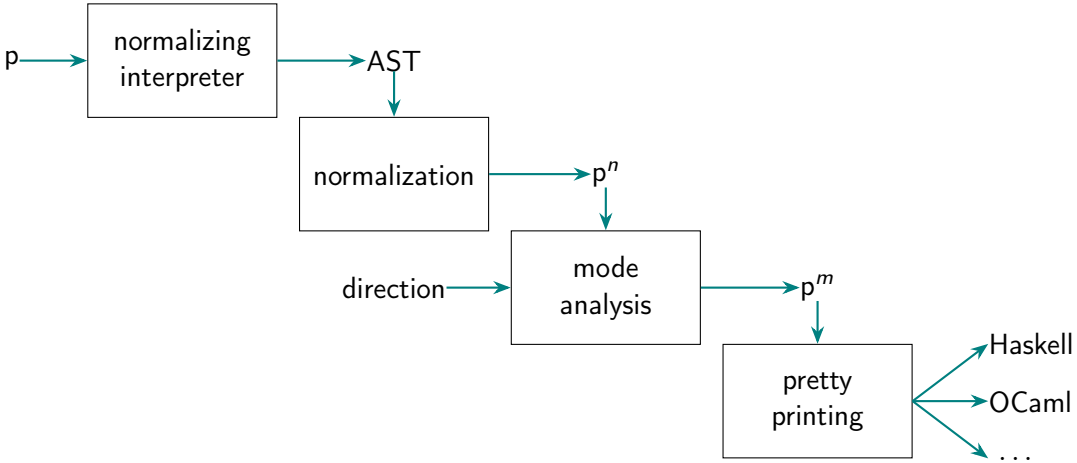
- Relational program execution
- Normalization
- Functional conversion[1]

[1] Re-using the ballpark of the old conversion

Functional Conversion



Functional Conversion



Old Functional Conversion Result

```
data Term
  = Z | S Term
  | Nil | Cons Term Term
  | Leaf | Node Term Term Term

balanceOO :: Stream Term → Stream Term → Stream (Term, Term)
balanceOO balancedO_x2 depthOI_x3 = do
  x1 ← delay (balancedO balancedO_x2 depthOI_x3);
  x2 ← delay (traversoIO x1);
   ...

depthOI :: Term → Stream Term → Stream Term
depthOI x1 depthOI_x3 = do
   ...
  x2 ← delay (depthOI x5 depthOI_x3);
  x4 ← delay (depthOI x6 depthOI_x3);
  x3 ← depthOI_x3;
   ...
```

New Functional Conversion Result

```
data Nat = Z | S Nat
data Tree elem = Leaf | Node (Tree elem) elem (Tree elem)

balanceOO :: Stream (Tree Nat, Tree Nat)
balanceOO = do
  x1 ← delay balancedoO
  x2 ← liftMaybe (traversoIO x1)
  ...
depthoOI :: Nat → Stream Tree
depthoOI x1 = do
  ...
  x2 ← delay (depthoOI x5)
  x4 ← delay (depthoOI x6)
  x3 ← runGen'
  ...
```